



# DCS 2026 Spring Lab03 – Code Review

---

TA : 陳孟頔    Advisor : Tian-Sheuan Chang

Date : 2026.03.24

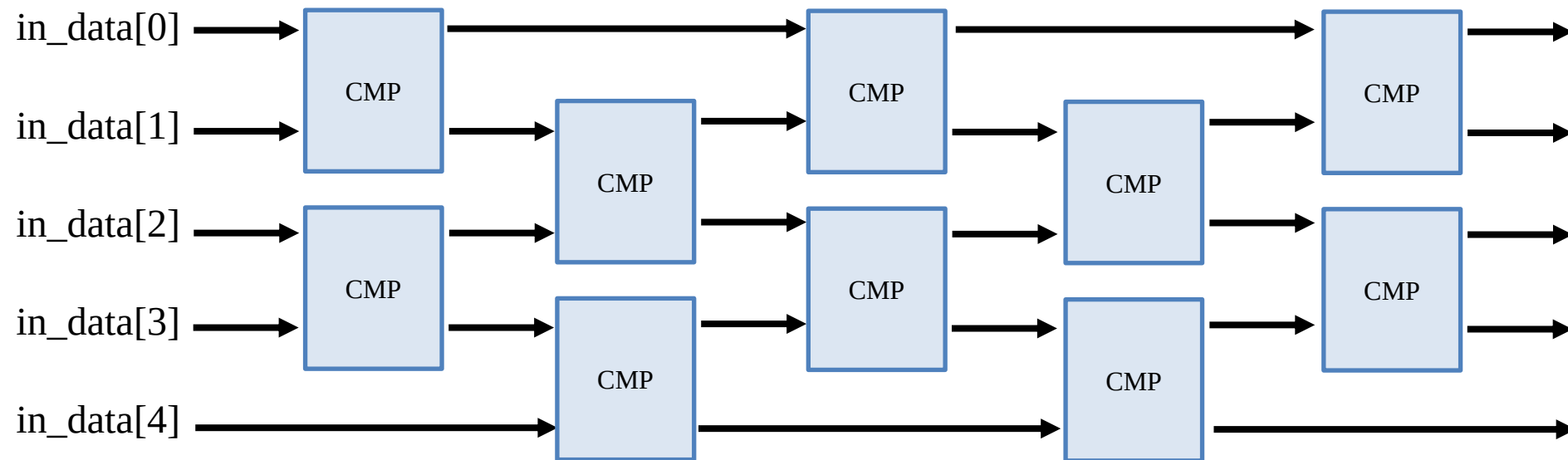
# Outline

---

- Lab Walkthrough
- Sequential vs. Computational Circuit
- Timing Violation

# Lab03: Sequential Sorting Network

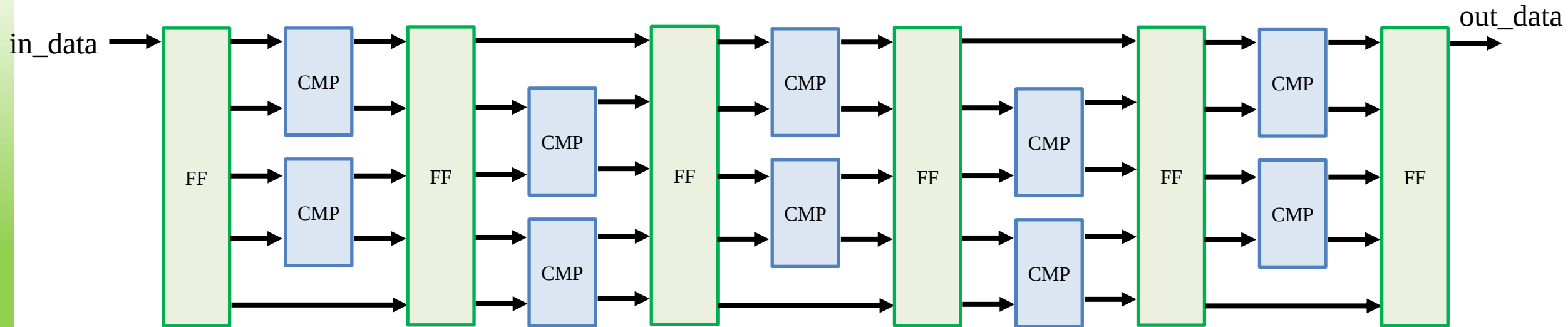
- Sorting Network:
  - Sort N numbers using Compare-and-Swap units.
  - Best for small N and ultra-low latency, parallel data.



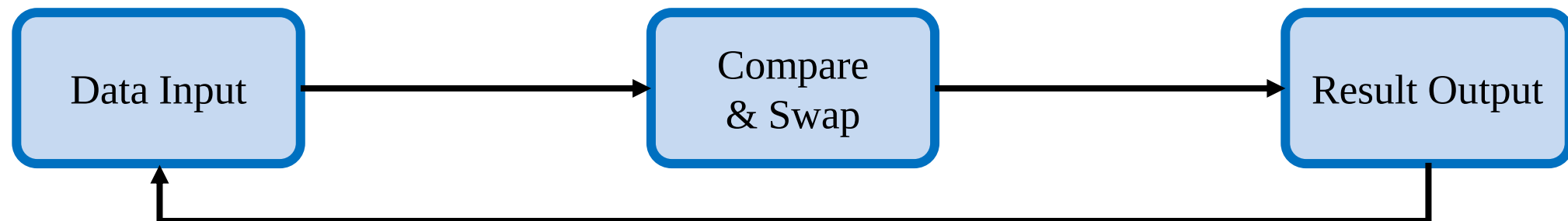
# Design Concept

- Block Diagram:

Use counter to control



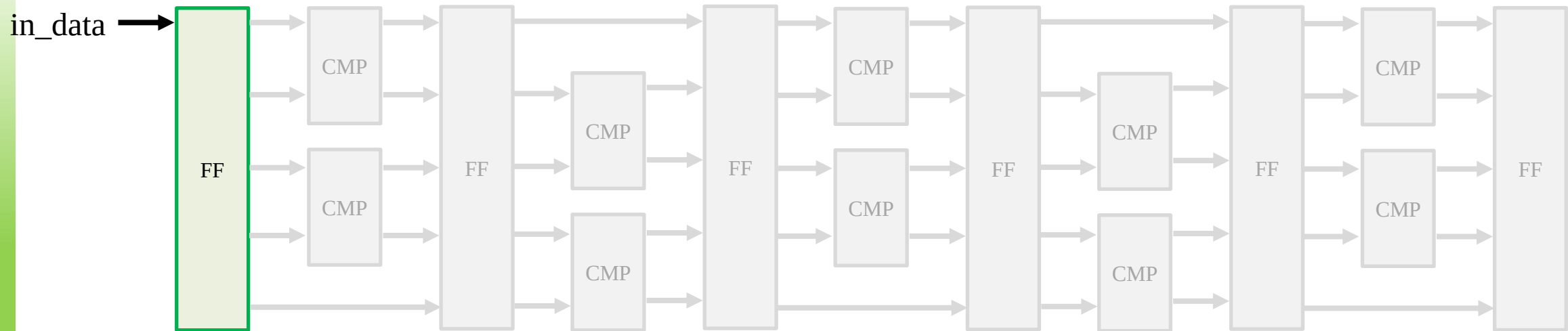
- State Diagram:



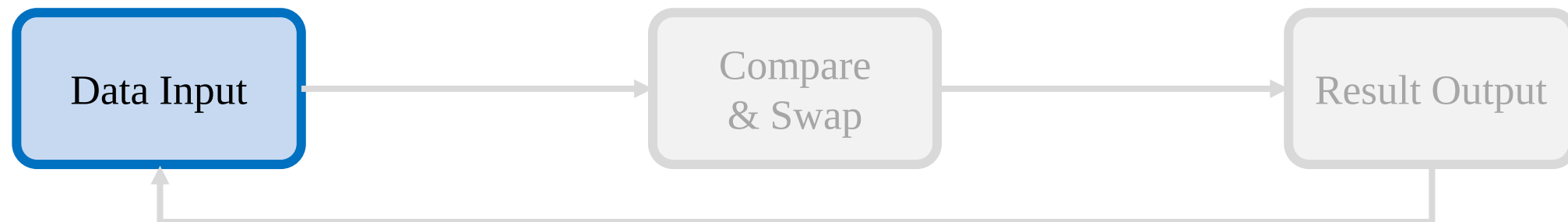
# Data Input Phase

- Block Diagram:

counter = 0~4



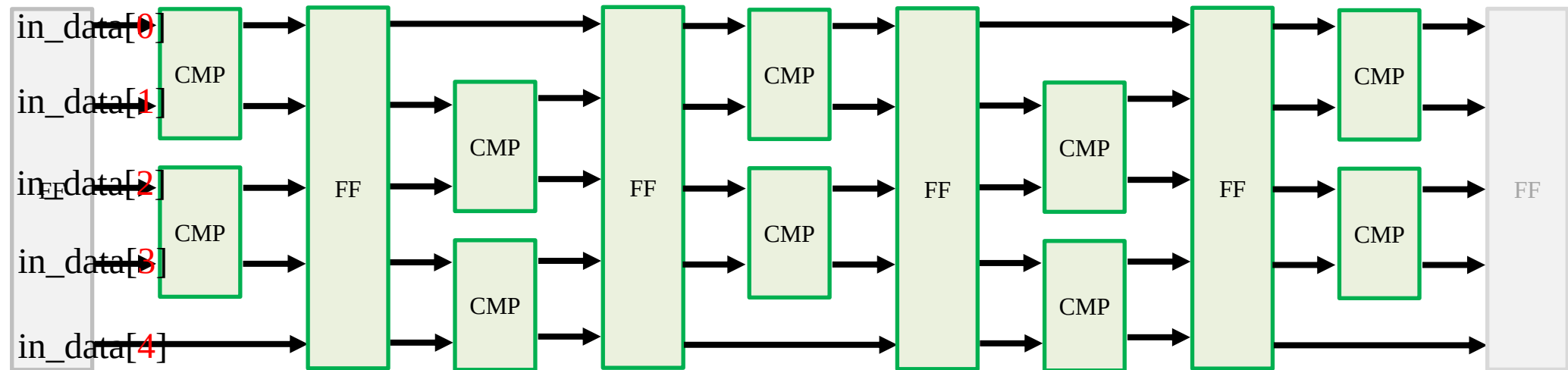
- State Diagram:



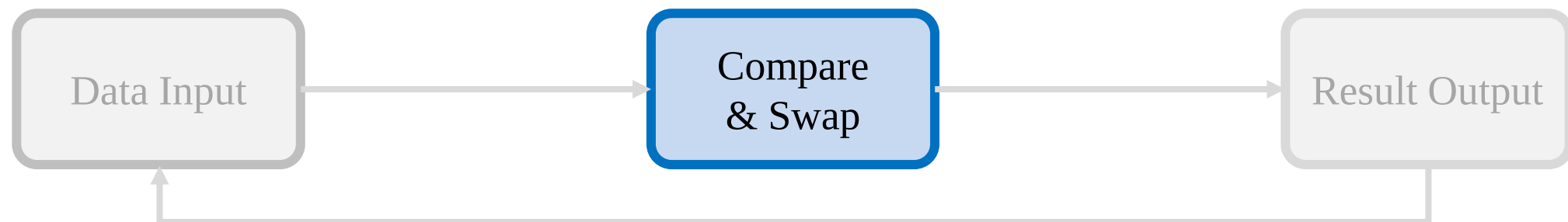
# Compare and Swap Phase

- Block Diagram:

counter = 5~9

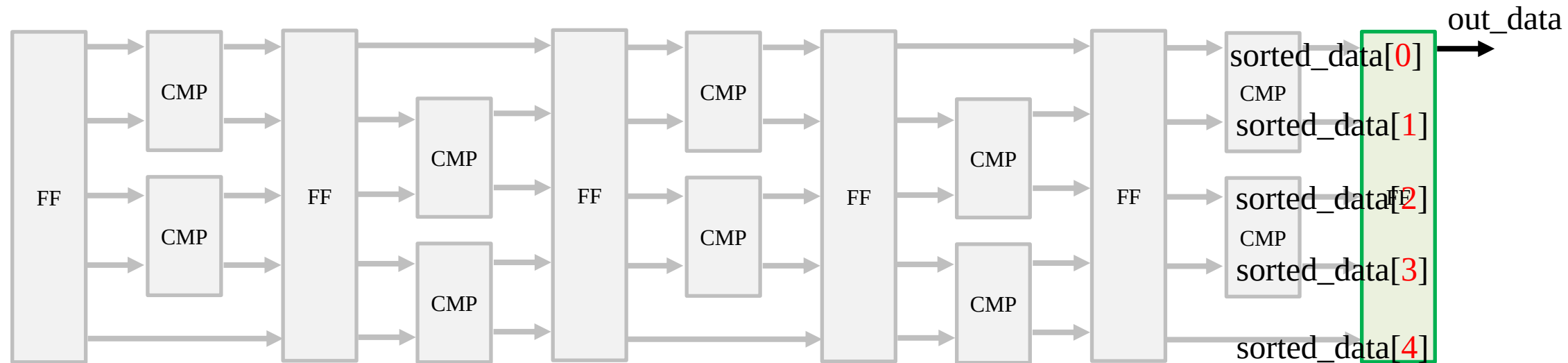


- State Diagram:



# Output Phase

- Block Diagram:

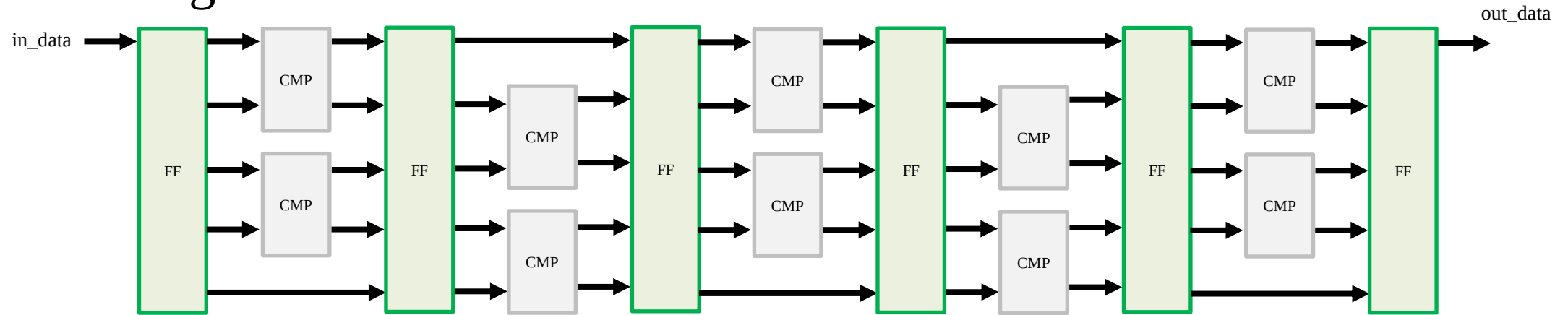


- State Diagram:



# Sequential Circuit

- Block Diagram:



```

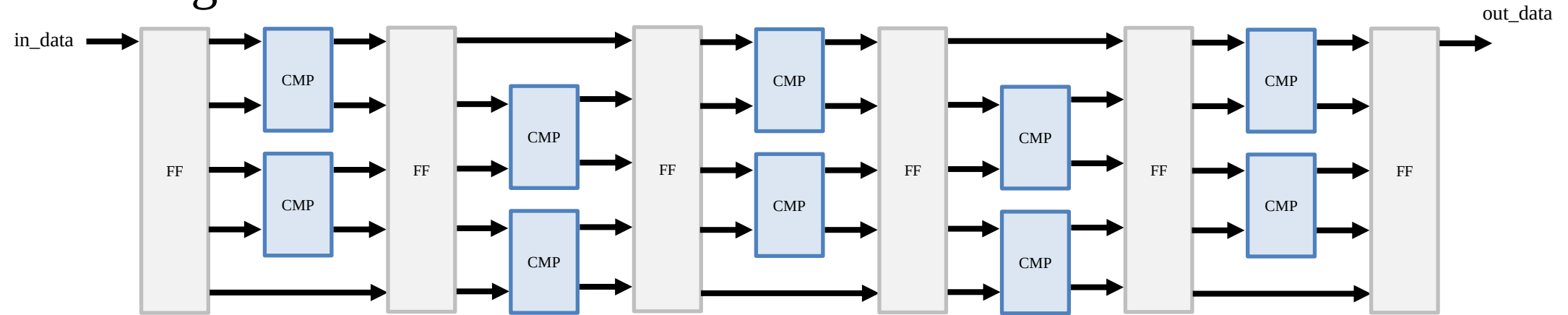
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        out_valid      <= 0;
        out_data       <= 0;
        in_data_cs     <= {0, 0, 0, 0, 0};
        stage1_cs      <= {0, 0, 0, 0, 0};
        stage2_cs      <= {0, 0, 0, 0, 0};
        stage3_cs      <= {0, 0, 0, 0, 0};
        stage4_cs      <= {0, 0, 0, 0, 0};
        ready_data_cs  <= {0, 0, 0, 0, 0};
        count_cs       <= 0;
    end else begin
        out_valid      <= out_valid_ns;
        out_data       <= out_data_ns;
        in_data_cs     <= in_data_ns;
        stage1_cs      <= stage1_ns;
        stage2_cs      <= stage2_ns;
        stage3_cs      <= stage3_ns;
        stage4_cs      <= stage4_ns;
        ready_data_cs  <= ready_data_ns;
        count_cs       <= count_ns;
    end
end

```



# Combinational Circuit

- Block Diagram:



```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        out_valid      <= 0;
        out_data       <= 0;
        in_data_cs     <= {0, 0, 0, 0, 0};
        stage1_cs      <= {0, 0, 0, 0, 0};
        stage2_cs      <= {0, 0, 0, 0, 0};
        stage3_cs      <= {0, 0, 0, 0, 0};
        stage4_cs      <= {0, 0, 0, 0, 0};
        ready_data_cs  <= {0, 0, 0, 0, 0};
        count_cs       <= 0;
    end else begin
        out_valid      <= out_valid_ns;
        out_data       <= out_data_ns;
        in_data_cs     <= in_data_ns;
        stage1_cs      <= stage1_ns;
        stage2_cs      <= stage2_ns;
        stage3_cs      <= stage3_ns;
        stage4_cs      <= stage4_ns;
        ready_data_cs  <= ready_data_ns;
        count_cs       <= count_ns;
    end
end
```

```
always @(*) begin
    in_data_ns = in_data_cs;
    if (in_valid) in_data_ns[count_cs] = in_data;
end
```

```
always @(*) begin
    if (count_cs >= 10 && count_cs <= 14) begin
        out_valid_ns = 1;
        out_data_ns = ready_data_cs[count_cs - 10];
    end else begin
        out_valid_ns = 0;
        out_data_ns = 0;
    end
end
```

```
CMP CMP1_1(.in1(in_data_cs[0]), .in2(in_data_cs[1]), .out1(stage1_ns[0]), .out2(stage1_ns[1]));
CMP CMP1_2(.in1(in_data_cs[2]), .in2(in_data_cs[3]), .out1(stage1_ns[2]), .out2(stage1_ns[3]));
assign stage1_ns[4] = in_data_cs[4];

CMP CMP2_1(.in1(stage1_cs[1]), .in2(stage1_cs[2]), .out1(stage2_ns[1]), .out2(stage2_ns[2]));
CMP CMP2_2(.in1(stage1_cs[3]), .in2(stage1_cs[4]), .out1(stage2_ns[3]), .out2(stage2_ns[4]));
assign stage2_ns[0] = stage1_cs[0];

CMP CMP3_1(.in1(stage2_cs[0]), .in2(stage2_cs[1]), .out1(stage3_ns[0]), .out2(stage3_ns[1]));
CMP CMP3_2(.in1(stage2_cs[2]), .in2(stage2_cs[3]), .out1(stage3_ns[2]), .out2(stage3_ns[3]));
assign stage3_ns[4] = stage2_cs[4];

CMP CMP4_1(.in1(stage3_cs[1]), .in2(stage3_cs[2]), .out1(stage4_ns[1]), .out2(stage4_ns[2]));
CMP CMP4_2(.in1(stage3_cs[3]), .in2(stage3_cs[4]), .out1(stage4_ns[3]), .out2(stage4_ns[4]));
assign stage4_ns[0] = stage3_cs[0];

CMP CMP5_1(.in1(stage4_cs[0]), .in2(stage4_cs[1]), .out1(ready_data_ns[0]), .out2(ready_data_ns[1]));
CMP CMP5_2(.in1(stage4_cs[2]), .in2(stage4_cs[3]), .out1(ready_data_ns[2]), .out2(ready_data_ns[3]));
assign ready_data_ns[4] = stage4_cs[4];
```

# Outline

---

- Lab Walkthrough
- Sequential vs. Computational Circuit
- Timing Violation

# Sequential v.s. Combinational in Verilog

|               | Sequential Circuit (D Flip-Flop)                                                                                                                                                                                                           | Combinational Circuit                                                                                                       |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Verilog Code  | <pre> always @ (posedge clk or negedge rst_n) begin     if(!rst_n) dff &lt;= 'b0;     else dff &lt;= dff_next; end  always_ff @ (posedge clk or negedge rst_n) begin     if(!rst_n) dff &lt;= 'b0;     else dff &lt;= dff_next; end </pre> | <pre> always @ * begin     sum = in1 + in2; end  always_comb begin     sum = in1 + in2; end  assign sum = in1 + in2; </pre> |
| Block Diagram |                                                                                                                                                                                                                                            |                                                                                                                             |

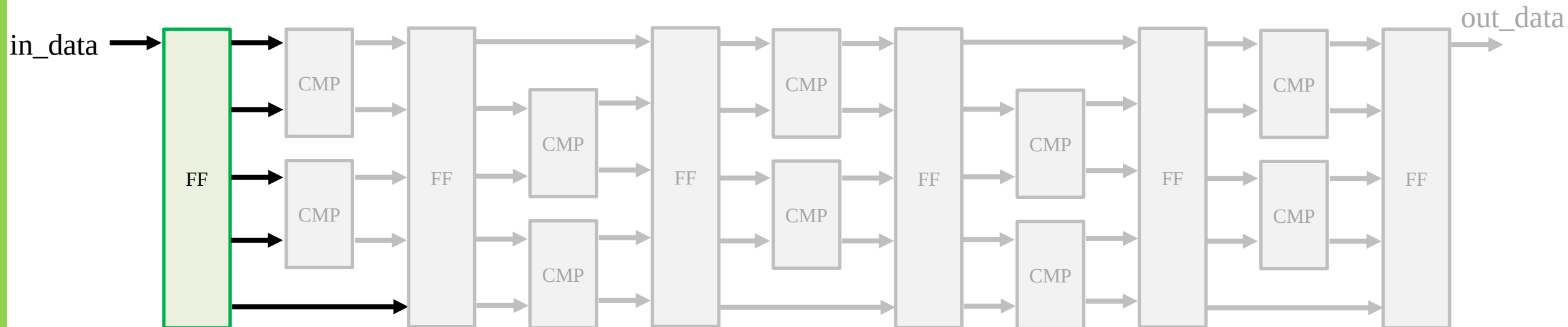
# Example 1 – Data Input

- Sequential Circuit:

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        in_data_cs    <= {0, 0, 0, 0, 0};
    end else begin
        in_data_cs    <= in_data_ns;
    end
end
end
```

- Combinational Circuit:

```
always @(*) begin
    in_data_ns = in_data_cs;
    if (in_valid) in_data_ns[count_cs] = in_data;
end
```



# Example 2 – Compare Stage 1

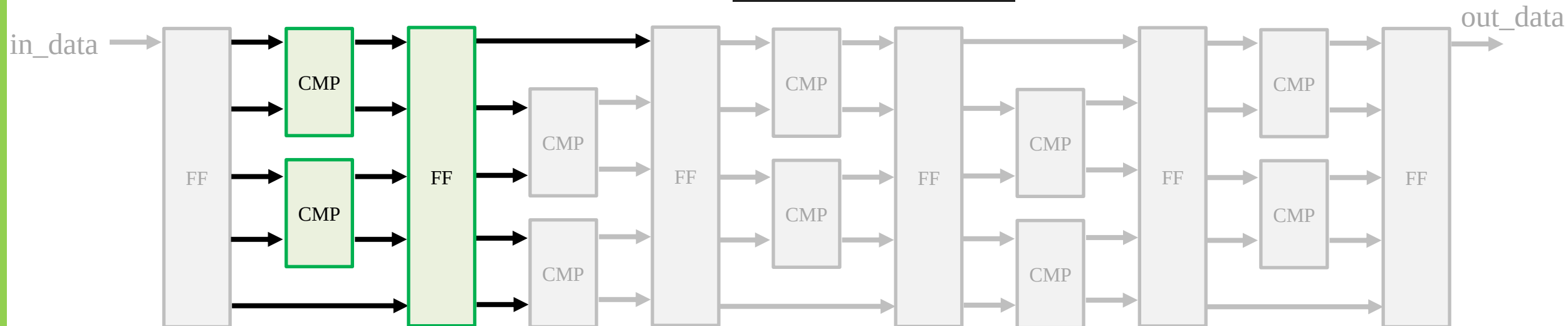
- Sequential Circuit:

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        stage1_cs      <= {0, 0, 0, 0, 0};
    end else begin
        stage1_cs      <= stage1_ns;
    end
end
end
```

- Combinational Circuit:

```
CMP CMP1_1(.in1(in_data_cs[0]), .in2(in_data_cs[1]), .out1(stage1_ns[0]), .out2(stage1_ns[1]));
CMP CMP1_2(.in1(in_data_cs[2]), .in2(in_data_cs[3]), .out1(stage1_ns[2]), .out2(stage1_ns[3]));
assign stage1_ns[4] = in_data_cs[4];
```

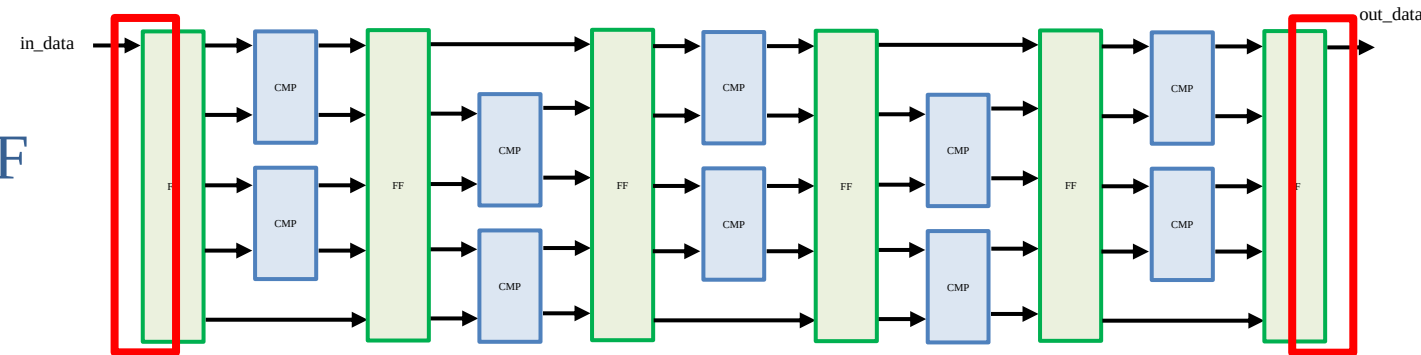
```
module CMP (
    input [5:0] in1,
    input [5:0] in2,
    output [5:0] out1,
    output [5:0] out2
);
    assign out1 = (in1 > in2) ? (in1) : (in2);
    assign out2 = (in1 > in2) ? (in2) : (in1);
endmodule
```



# Reminders

- Naming Consistency:
  - Next state logic  $\rightarrow$  **<logic>\_ns**
  - Current state logic  $\rightarrow$  **<logic>\_cs**
- Sequential Logic:
  - Store input & output data in DFF
- Combinational Logic:
  - Always use default or complete if-else statement

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        in_data_cs    <= {0, 0, 0, 0, 0};
    end else begin
        in_data_cs    <= in_data_ns;
    end
end
end
```



```
always @(*) begin
    if (count_cs == 14) begin
        count_ns = 0;
    end else if (in_valid && count_cs == 0) begin
        count_ns = 1;
    end else if (count_cs > 0) begin
        count_ns = count_cs + 1;
    end else begin
        count_ns = 0;
    end
end
end
```

```
case (cs)
    CAL: begin
        if (cnt_cal_1_cs == 6) cnt_cal_1_ns = 1;
        else cnt_cal_1_ns = cnt_cal_1_cs + 1;
    end
    default: cnt_cal_1_ns = 1;
endcase
```

# Outline

---

- Lab Walkthrough
- Sequential vs. Computational Circuit
- Timing Violation

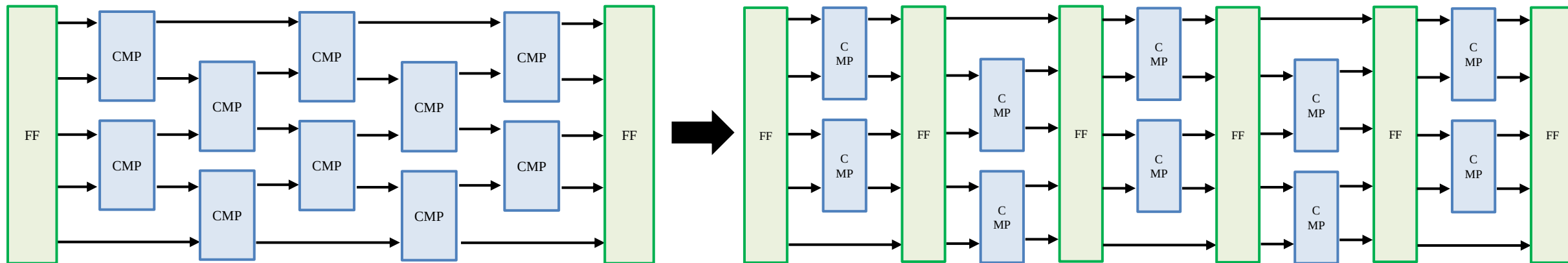
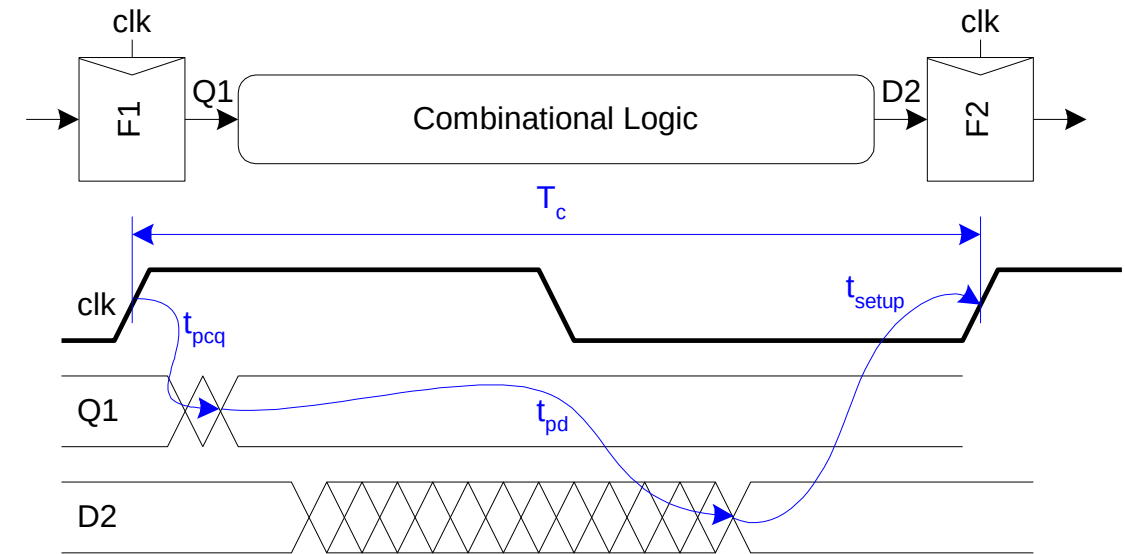
# Timing Violation

| Point                             | Incr  | Path   |
|-----------------------------------|-------|--------|
| -----                             |       |        |
| clock clk (rise edge)             | 0.00  | 0.00   |
| clock network delay (ideal)       | 0.00  | 0.00   |
| cnt_cal_1_cs_reg_2_/CK (DFFRHQX4) | 0.00  | 0.00 r |
| cnt_cal_1_cs_reg_2_/Q (DFFRHQX4)  | 0.36  | 0.36 r |
| U4559/Y (INVX4)                   | 0.10  | 0.47 f |
| U2391/Y (INVX4)                   | 0.08  | 0.55 r |
| U4136/Y (NOR2X4)                  | 0.09  | 0.63 f |
| U4541/Y (INVX8)                   | 0.12  | 0.76 r |
| U4458/Y (INVX4)                   | 0.07  | 0.83 f |
| U2918/Y (INVX8)                   | 0.07  | 0.90 r |
| U2920/Y (INVX8)                   | 0.06  | 0.96 f |
| U2947/Y (AOI22X1)                 | 0.24  | 1.20 r |
| U4438/Y (NAND4X2)                 | 0.15  | 1.35 f |
| U4406/Y (AOI22X2)                 | 0.24  | 1.59 r |
| U5129/Y (NAND4X2)                 | 0.13  | 1.72 f |
| U4256/CO (ADDFHX2)                | 0.47  | 2.19 f |
| U5183/CO (ADDFHX4)                | 0.48  | 2.67 f |
| U2647/S (ADDFHX4)                 | 0.39  | 3.06 f |
| U4080/Y (NOR2X4)                  | 0.12  | 3.19 r |
| U5229/Y (OAI21X2)                 | 0.09  | 3.28 f |
| U5232/Y (AOI21X1)                 | 0.15  | 3.43 r |
| U5233/Y (OAI21X1)                 | 0.14  | 3.57 f |
| U5234/Y (INVX3)                   | 0.12  | 3.69 r |
| U2816/Y (INVX4)                   | 0.07  | 3.75 f |
| U2815/Y (INVX8)                   | 0.08  | 3.83 r |
| U4199/Y (MXI2X4)                  | 0.16  | 4.00 r |
| U4198/Y (XOR2X4)                  | 0.29  | 4.28 r |
| U4380/Y (NAND2X2)                 | 0.07  | 4.35 f |
| U4369/Y (NAND4X2)                 | 0.12  | 4.47 r |
| U4365/Y (NAND3X2)                 | 0.09  | 4.57 f |
| U4363/Y (NAND3X2)                 | 0.10  | 4.66 r |
| U2411/Y (NAND2X2)                 | 0.07  | 4.74 f |
| U2410/Y (NAND2X2)                 | 0.07  | 4.81 r |
| U4290/Y (NAND2X1)                 | 0.06  | 4.87 f |
| out_data_reg/D (DFFRX4)           | 0.00  | 4.87 f |
| data arrival time                 |       | 4.87   |
|                                   |       |        |
| clock clk (rise edge)             | 5.00  | 5.00   |
| clock network delay (ideal)       | 0.00  | 5.00   |
| clock uncertainty                 | -0.10 | 4.90   |
| out_data_reg/CK (DFFRX4)          | 0.00  | 4.90 r |
| library setup time                | -0.25 | 4.65   |
| data required time                |       | 4.65   |
| -----                             |       |        |
| data required time                |       | 4.65   |
| data arrival time                 |       | -4.87  |
| -----                             |       |        |
| slack (VIOLATED)                  |       | -0.22  |



# Timing Violation

- Result of Timing Violation:
  - Critical combinational path too long
- Solution:
  - Store intermediate value



Thank you.  
Any questions?